**ENTERPRISE DATA ENGINEERING CAPSTONE PROJECT**

# ENTERPRISE HUMAN CAPITAL MANAGEMENT DATA PLATFORM (EHCMDP)

*Building a Unified Workforce & HR Analytics Platform*



| | |
|---|---|
| Team members(UC17) | Ankita Yeleswarapu — 26MML1017<br>Likisha Ulhas Kharche — 26MML1032<br>Madhumitha M — 26MML1031 |

# Contents

1. Abstract ..... 3
2. Introduction ..... 4
3. Problem Statement and Business Need ..... 5
4. Objectives, Scope and Deliverables ..... 6
5. Stakeholders and Requirements ..... 8
6. Methodology and Sprint Plan ..... 10
7. Solution Architecture ..... 12
8. Source Systems and Data Inventory ..... 15
9. Data Ingestion and Processing Design ..... 17
10. Data Model and Storage Design ..... 19
11. Data Lineage and Metadata Governance ..... 22
12. Data Quality and Reconciliation ..... 25
13. Security, Privacy and Access Control ..... 30
14. Analytics and Power BI Consumption ..... 32
15. Testing and Validation Strategy ..... 35
16. Operations and Change Management ..... 37
17. Risks, Assumptions and Limitations ..... 39
18. Expected Outcomes and Success Criteria ..... 41
19. Conclusion ..... 42
**Appendix A. Source-to-Target Mapping Register** ..... 43
**Appendix B. Data Quality Rule Catalogue** ..... 45
**Appendix C. Business Glossary and KPI Definitions** ..... 46
**Appendix D. Lineage Evidence Checklist** ..... 47

# 1. Abstract

ABC Global Enterprises Ltd. operates in a multi-country, multi-business-unit environment with a workforce exceeding 75,000 employees. Human capital information is typically distributed across operational systems such as HRMS, recruitment, payroll, attendance, leave, learning, performance management, employee self-service, identity and access management, and workforce planning. Fragmented sources can create inconsistent definitions, duplicated records, delayed reporting and limited visibility into how a dashboard value was produced.

The Enterprise Human Capital Data Platform (EHCMDP) project defines an integrated data engineering solution to consolidate workforce data into a governed analytical platform. The design uses Pentaho Data Integration (Spoon) for ingestion and transformation orchestration, PostgreSQL for relational storage and curated analytical datasets, Python/Pandas for profiling and validation support, Git/GitHub for version control, and Power BI for reporting and decision support. The report documents end-to-end data movement, logical warehouse structures, source-to-target mapping, field transformation standards, quality controls, metadata, privacy safeguards and operational lineage.

The report presents a practical capstone design aligned with four project stages: initiation, ingestion, profiling/storage, and governance/lineage/analytics. Architectural and data-model diagrams are logical design views. Where a physical job name, live execution statistic or production configuration is not established by the project brief, the report defines the required implementation standard without representing it as an observed result.

## Keywords

Data engineering; human capital management; ETL/ELT; Pentaho PDI; PostgreSQL; data lineage; data quality; metadata governance; workforce analytics; Power BI.

# 2. Introduction

## 2.1 Background

Human capital management requires timely, reliable information about employee populations, organizational structures, employment status, hiring, payroll, attendance, learning and performance. When these records are maintained in separate applications, each system may use different identifiers, date conventions, status values and update cycles. Analytics teams must resolve these differences before producing consistent reports.

EHCMDP establishes a common data pipeline and governance framework so that source records can be traced through ingestion, standardization, storage and analytical presentation. The platform is designed around traceability and controlled transformation, not merely copying files into a database.

## 2.2 Project context

The UC17 brief frames ABC Global Enterprises Ltd. as a large enterprise with more than 75,000 employees distributed across multiple countries and business units. The capstone is organized into Sprint 0 (project initiation), Sprint 1 (data ingestion), Sprint 2 (data profiling and storage), and Sprint 3 (governance, lineage and analytics).

## 2.3 Report organization

The report moves from business need and requirements to architecture, data inventory, processing, warehouse design, lineage, data quality, security, reporting, testing, operational controls and delivery criteria. Appendices provide mapping and control registers that can be used during implementation and review.

# 3. Problem Statement and Business Need

The central problem is the lack of a single, consistently governed analytical view across multiple HR source domains. A report can be technically correct at the dashboard level yet remain difficult to trust if the source record, transformation rule, refresh run or data-quality outcome cannot be traced.

## 3.1 Key challenges

- Data silos: employee and workforce information is spread across function-specific systems.
- Identifier inconsistency: the same person or organization may be represented by different source keys.
- Semantic inconsistency: terms such as active employee, headcount, hire date and department may be interpreted differently.
- Quality risk: missing keys, duplicate records, invalid dates, inconsistent codes and incomplete extracts can flow into analytics.
- Limited traceability: users may not know which source, transformation or load run produced a metric.
- Privacy risk: HR data can contain personal, compensation, attendance and performance information requiring restricted handling.
- Change impact: source schema changes can break mappings, downstream tables, and Power BI measures if they are not assessed.

## 3.2 Business need

A unified platform should provide governed datasets, reusable metric definitions, reliable refresh processes, reconciliation controls and clear lineage from source to report. This supports workforce planning and management reporting while improving transparency for technical support and audit review.

# 4. Objectives, Scope and Deliverables

## 4.1 Objectives

- Define an end-to-end logical architecture for integrating HR-related data domains.
- Establish a repeatable ingestion and transformation pattern using Pentaho PDI.
- Store standardized, relational and analytics-ready data in PostgreSQL.
- Apply Python/Pandas-assisted profiling and validation where useful.
- Define source-to-target and field-level mappings with explicit transformation rules.
- Design a logical dimensional model to support workforce reporting.
- Document system-, dataset-, field- and run-level lineage.
- Define data quality, reconciliation, quarantine and reprocessing controls.
- Specify access control, privacy, masking, retention and audit requirements.
- Deliver a clear handover standard for dashboards, metadata and operational support.

## 4.2 In scope

| In-scope area | Coverage |
|---|---|
| HR data domains | HRMS/employee master, recruitment, payroll, attendance, leave, LMS, performance management, ESS, IAM and workforce planning |
| Data integration | Extract/load patterns, batch identifiers, staging, schema checks, transformation and error handling |
| Data storage | Raw/staging and curated logical zones, relational keys, facts and dimensions |
| Governance | Business glossary, lineage metadata, mapping versioning, change impact and ownership |
| Quality | Completeness, validity, uniqueness, referential integrity, reconciliation and exception disposition |
| Analytics | Power BI semantic model and workforce KPI definitions |
| Project management | Sprint-aligned deliverables, testing strategy, risk controls and acceptance criteria |

## 4.3 Out of scope

- Replacing the enterprise HR source applications themselves.
- Making automated employment, compensation or performance decisions without appropriate human governance.
- Claiming real-time streaming, enterprise production deployment or measured service-level performance unless separately evidenced.
- Defining jurisdiction-specific legal retention periods without approval from the organization's legal/privacy stakeholders.

## 4.4 Main deliverables

| **Deliverable** | **Description** |
|---|---|
| Architecture design | Logical flow from HR source systems through ingestion, storage, governance and BI |
| Ingestion design | Repeatable source extraction, batch tracking, schema validation and failure handling |
| Data inventory | Source domains, candidate entities, keys, sensitive fields and target subject areas |
| Warehouse design | Logical dimensions/facts and conformed key standards |
| Lineage report | System, dataset, field and run-level traceability with source-to-target mappings |
| Quality framework | Rule catalogue, reconciliation checks, quarantine and reprocessing controls |
| Analytics specification | Metric definitions and consumption rules for Power BI |
| Testing and operations | Test cases, release checks, monitoring metadata and acceptance criteria |

# 5. Stakeholders and Requirements

## 5.1 Stakeholder groups

| Stakeholder | Interest / responsibility |
|---|---|
| HR business owners | Define workforce terminology, KPI meaning, acceptable exceptions and reporting needs |
| Source system owners | Explain source schemas, identifiers, extraction options and source change notices |
| Data engineering team | Build and maintain Pentaho jobs, transformations, PostgreSQL structures and metadata |
| Data quality steward | Review profiling results, data exceptions, reconciliations and corrective actions |
| Database administrator | Manage PostgreSQL security, backups, performance, access and recovery |
| BI developer / analyst | Build Power BI model, measures, reports and validation comparisons |
| Security / privacy stakeholders | Approve access, masking, retention, sharing and sensitive-data controls |
| Project guide / evaluator | Review scope, design rationale, evidence, testing and project deliverables |

## 5.2 Functional requirements

- FR-01: ingest data from the defined HR domains through controlled, repeatable jobs.
- FR-02: retain source identity and batch/run metadata for every load.
- FR-03: validate file/schema expectations before data is promoted.
- FR-04: standardize identifiers, dates, codes, null handling and reference values.
- FR-05: load curated relational datasets using defined primary/foreign-key rules.
- FR-06: record rejected rows and reasons without silently discarding exceptions.
- FR-07: reconcile source, accepted, rejected and target record counts.
- FR-08: expose documented workforce datasets and KPI definitions for Power BI.

- FR-09: record mapping/job versions and assess downstream impact before changes.
- FR-10: enforce role-based access and limit exposure of sensitive employee attributes.

## 5.3 Non-functional requirements

| **Requirement** | **Design standard** |
|---|---|
| Reliability | Restartable jobs, explicit run status, error capture and controlled reprocessing |
| Traceability | Source-to-target mapping and run-level identifiers retained end to end |
| Data quality | Defined rules, severities, thresholds and exception disposition |
| Security | Least privilege, restricted sensitive columns, credential protection and audit logs |
| Maintainability | Version-controlled jobs, SQL, mapping specifications and release notes |
| Scalability | Partitioning/indexing and incremental processing considered as volume and workload require |
| Usability | Consistent metric definitions and understandable dashboard labels |
| Portability | Environment-specific configuration separated from transformation logic |

# 6. Methodology and Sprint Plan

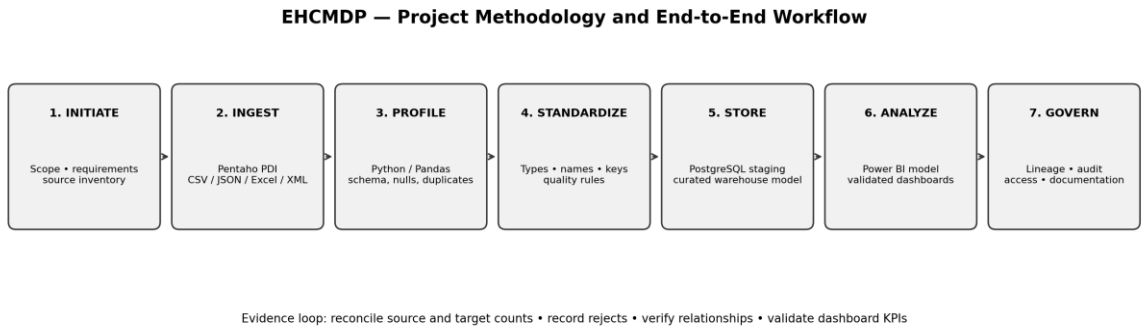
The project follows a staged data engineering lifecycle. Each sprint produces artifacts that are prerequisites for the next stage; the plan emphasizes traceability between requirements, implementation, testing and review.

| Sprint | Focus | Primary activities | Exit evidence |
|---|---|---|---|
| Sprint 0 — Initiation | Scope and planning | Confirm business questions, data domains, roles, risks, source inventory and toolchain | Approved scope, requirements register, architecture outline and sprint plan |
| Sprint 1 — Ingestion | Source acquisition | Inspect source formats, build extraction/load jobs, establish landing/staging and run identifiers | Repeatable ingestion runs, source inventory and ingestion logs |
| Sprint 2 — Profiling and storage | Quality and persistence | Profile nulls/duplicates/ranges, standardize fields, define keys, create curated PostgreSQL structures | Profiling results, schema/mapping definitions, quality checks and load reconciliation |
| Sprint 3 — Governance, lineage and analytics | Trust and consumption | Complete lineage, glossary, access rules, quality exception process, dimensional model and Power BI definitions | Lineage register, KPI catalogue, test evidence, security checklist and report-ready analytics |

## 6.1 Delivery approach

1. Discover: document source domain, business owner, extraction method and sensitivity.
2. Profile: inspect schema, volume, null patterns, uniqueness, value distributions and date ranges.
3. Design: approve mappings, target grain, key strategy, history rules and quality gates.
4. Build: implement versioned Pentaho jobs and PostgreSQL structures.
5. Validate: compare counts and aggregates, test exception handling and reconcile report outputs.
6. Release: review security, run logs, downstream impact, rollback and support documentation.

Figure 1. EHCMDP project methodology and end-to-end workflow.




## 6.2 Version control

Git/GitHub should store transformation specifications, SQL DDL/DML, Python profiling scripts, data-quality rules, configuration templates, dashboard definitions where exportable, and documentation. Secrets, production credentials and unmasked personal data must not be committed to the repository.

# 7. Solution Architecture

The architecture diagram presents a flow from input source datasets to the data-ingestion layer implemented with Pentaho Data Integration. ETL pipelines support the movement and processing of records. The storage layers organize data as raw, cleaned and standardized, and curated data. Metadata and data distribution support controlled handling across these layers, while data governance and lineage provide traceability. The final analytics and consumption layer supplies dashboards, reports and user-facing outputs. In this project, PostgreSQL is the relational storage platform, Python/Pandas supports profiling and validation, Git/GitHub supports version control, and Power BI is the reporting tool.

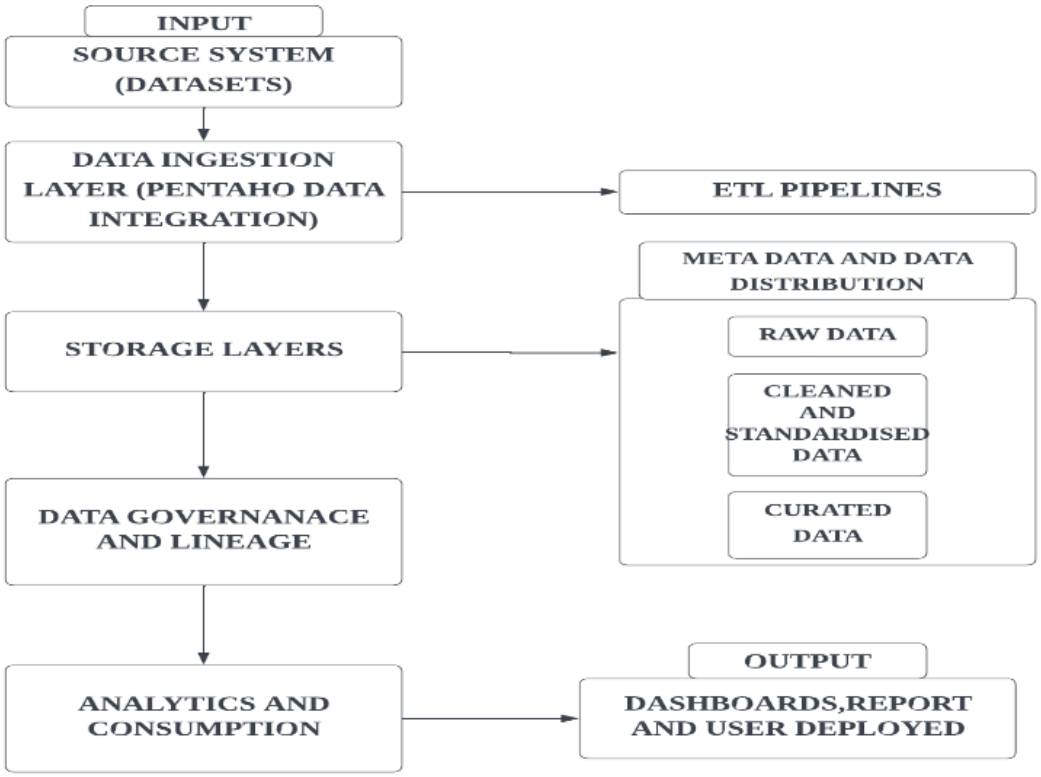



Figure 2. EHCMDP logical architecture and principal data movement.

## 7.1 Architecture layers

| Layer | Role | Key controls |
|---|---|---|
| Input source systems | Provide HR and workforce datasets in source formats | Source owner, approved scope, snapshot timestamp and sensitivity classification |
| Data-ingestion layer | Pentaho Data Integration reads source datasets and initiates controlled loads | Job version, parameters, run ID, restart strategy and failure logging |

| | | |
|---|---|---|
| ETL pipelines | Move and transform records between source and storage layers | Versioned mappings, schema checks, error capture and repeatable execution |
| Storage layers — raw data | Retain source-shaped data for controlled processing and replay | Batch ID, load timestamp, schema validation and restricted access |
| Storage layers — cleaned and standardized data | Apply type, naming, code, key and quality standardization | Validation rules, transformation version, reject reasons and quality outcomes |
| Storage layers — curated data | Provide prepared subject-area datasets for analytical use | Documented keys, constraints, reconciliation and access grants |
| Metadata and data distribution | Describe assets and support controlled movement/availability of data | Metadata records, source-to-target mappings, lineage references and approved distribution |
| Data governance and lineage | Connect controls and traceability across the pipeline | Glossary, lineage, access logs, change records and quality evidence |
| Analytics and consumption | Power BI semantic model, dashboards, reports and measures | Certified definitions, refresh validation and row/column security where needed |

## 7.2 Design principles

- Preserve source traceability; do not overwrite source identifiers with warehouse keys.
- Separate raw/staging data from standardized and curated outputs.
- Use explicit schemas and documented transformations rather than hidden spreadsheet logic.
- Make loads observable, restartable and reconcilable.
- Apply the least-privilege principle to source, staging, warehouse and BI access.
- Keep logical design distinct from physical implementation evidence.

## 7.3 Logical data flow

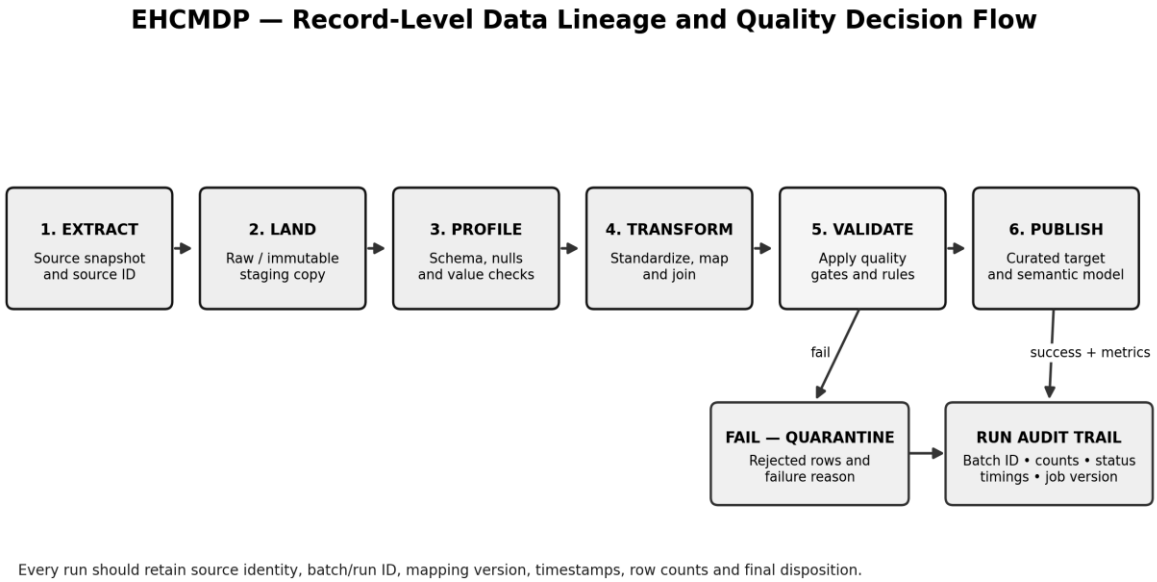



Figure 3. Record-level lineage through profiling, transformation, quality decisions and publication.

The successful path promotes accepted records into curated tables and the BI model. The failure path records rejected data and the reason for rejection, enabling review and controlled reprocessing. Every run should retain a stable identifier linking the source extract, transformation version, target load and quality results.

# 8. Source Systems and Data Inventory

The following inventory is the logical source-domain baseline derived from the UC17 project context. The exact application product, physical schema and extraction mechanism must be recorded from the actual source environment during implementation; the domain-level mapping below does not assume an unverified vendor product.

| Source domain | Representative entities / fields | Likely target subject area | Sensitive-data considerations |
|---|---|---|---|
| HRMS / Employee Master | Employee ID, name, hire date, employment status, manager ID, job/grade, department | Employee and employment dimensions; workforce snapshot | Direct identifiers, employment history |
| Recruitment | Requisition ID, candidate ID, application date, stage, offer outcome, hire reference | Recruitment fact / hiring funnel | Candidate personal data; restrict access |
| Payroll | Payroll period, employee ID, earnings, deductions, currency, pay status | Payroll fact / compensation subject area | Highly sensitive compensation and bank-related attributes |
| Attendance | Employee ID, work date, hours, attendance code, shift | Attendance fact | Work patterns and potentially sensitive absence indicators |
| Leave | Employee ID, leave type, start/end date, duration, approval status | Leave fact | Absence reasons may reveal sensitive information |
| Learning Management System | Employee ID, course ID, enrollment, completion date, status | Learning fact and course dimension | Training records and employee development |
| Performance Management | Employee ID, review period, rating, goal status | Performance subject area | Highly restricted performance information |
| Employee Self-Service | Employee ID, profile changes, requests, approvals | Change/request event subject area | May contain personal contact and profile data |
| Identity & Access Management | User ID, employee reference, role, access status | Access governance reference | Security-sensitive identity and role data |
| Workforce Planning | Organization, role, planned FTE, budget period, location | Planning fact and organization dimension | Planning assumptions and business-sensitive data |

## 8.1 Data acquisition contract

- Source owner and approved purpose of use are recorded for each dataset.
- Each extract carries a source identifier, extraction timestamp and schema/version reference.
- Files are checked for presence, readability, expected delimiter/encoding and required columns.
- Source timezone, date format, currency and code-set assumptions are documented.
- Late, duplicate, partial or corrected source deliveries are detected and handled by an explicit policy.

## 8.2 Data classification

| Class | Examples | Handling baseline |
|---|---|---|
| General operational | Non-sensitive code lists, reporting calendars | Controlled internal access; maintain integrity and change history |
| Confidential workforce | Employee status, organization, role and work history | Role-based access; minimize columns; restrict export |
| Highly sensitive HR | Payroll amounts, performance ratings, leave reasons, identity/access details | Need-to-know access, masking/tokenization where feasible, audit access and approved retention |

# 9. Data Ingestion and Processing Design

## 9.1 Ingestion pattern

Batch ingestion is the baseline design for periodic HR extracts because it supports repeatable snapshots, reconciliation and controlled reruns. A source may support incremental extraction, but the watermark column, late-arriving-data policy and deletion handling must be defined per source before incremental mode is enabled. Hybrid or streaming ingestion is not assumed unless the source and business requirement justify it.

## 9.2 Standard processing sequence

1. Create a run record and assign a unique batch/run identifier.
2. Read source metadata and validate file/object existence and schema compatibility.
3. Land source data with extraction and ingestion timestamps.
4. Profile key fields and apply schema, type, null, uniqueness and reference checks.
5. Transform values using versioned mappings and standardized data types.
6. Resolve business keys and apply deduplication and history rules.
7. Write accepted records to curated PostgreSQL targets and rejected records to a quarantine area.
8. Reconcile counts and key aggregates; mark the run successful, warning or failed.
9. Publish or refresh the Power BI semantic model only after the curated load passes its release gates.

## 9.3 Idempotency and restartability

A rerun for the same source batch should not create unintended duplicates. The implementation should define a natural-key or batch-key strategy, use transactional loading where practical, record the last completed step and distinguish a retry from a new source delivery. Reprocessing must be traceable to the original failure and subsequent correction.

## 9.4 Full and incremental loads

| Load mode | Use case | Required controls |
|---|---|---|

| | | |
|---|---|---|
| Full snapshot | Source provides complete current population or dataset | Snapshot timestamp, source count, duplicate check, target replacement/merge policy and rollback plan |
| Incremental | Source exposes a reliable changed-at timestamp, sequence or CDC feed | Watermark persistence, overlap window, late-arriving changes, delete/tombstone policy and duplicate-safe merge |
| Correction / replay | Source reissues an earlier period or corrected extract | Original batch reference, correction reason, deterministic replay and before/after reconciliation |

## 9.5 Error handling

Errors should be classified as technical (connection, permission, file parsing, database constraint), structural (missing/renamed columns or incompatible schema), or data-level (invalid code, missing key, duplicate natural key or impossible date). Critical structural and technical failures should stop promotion. Data-level failures should be quarantined or rejected according to severity and approved business rules.

## 9.6 Processing metadata

| **Metadata field** | **Purpose** |
|---|---|
| run_id / batch_id | Uniquely identifies one execution and its source delivery |
| source_system / source_object | Identifies the origin of the records |
| source_extract_timestamp | Indicates when the source snapshot was created |
| job_name / job_version | Identifies transformation logic used for the run |
| start_timestamp / end_timestamp | Supports duration analysis and operational support |
| source_count / accepted_count / rejected_count / target_count | Supports load completeness and reconciliation |
| run_status / error_code / error_message | Explains execution outcome and failure context |
| mapping_version / target_object | Connects field mapping and target table version |

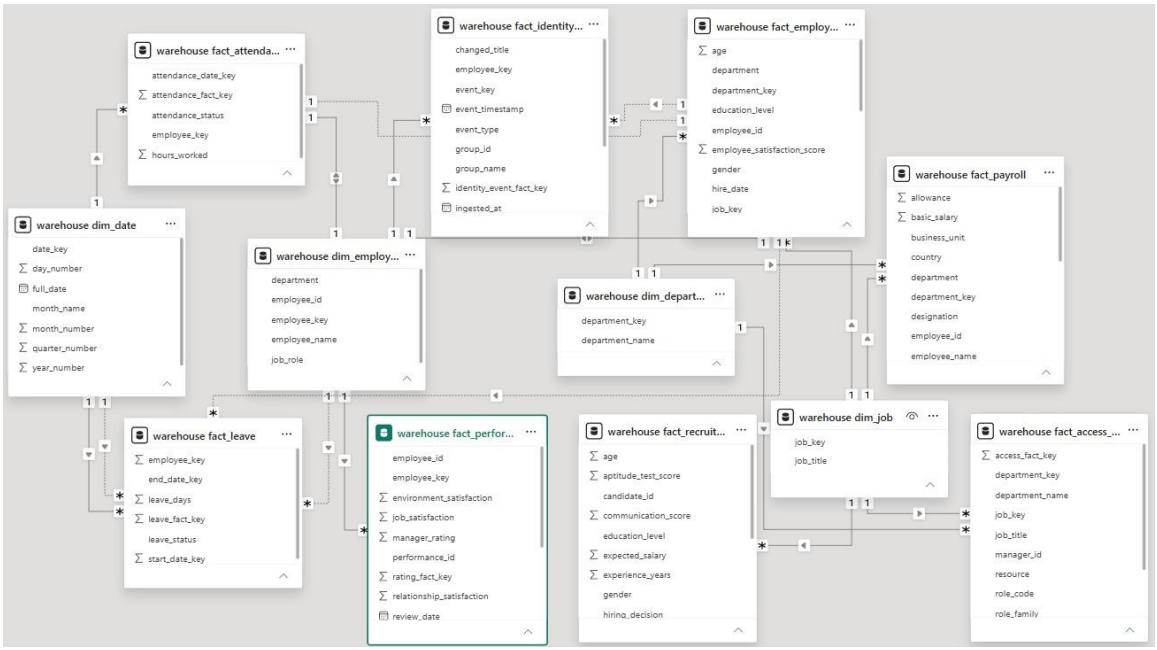
# 10. Data Model and Storage Design

## 10.1 Storage zones

| Zone | Purpose | Design rule |
|---|---|---|
| Landing / raw | Preserve source-shaped extracts for replay and investigation | Append or version source deliveries; restrict access; never silently mutate original source payload |
| Staging / standardized | Normalize types, column names, code values and keys | Record transformation version; validate required fields; keep rejected rows traceable |
| Curated / analytical | Provide conformed subject-area tables for reporting | Enforce documented keys, relationships, metric grain and quality gates |
| Semantic / BI | Present approved measures and reporting dimensions | Use certified definitions, controlled refresh and appropriate row/column access |

## 10.2 Logical dimensional model

*Figure 4. Power BI model relationships using the actual warehouse table names shown in the supplied screenshot.*




Naming convention note: the screenshot itself uses warehouse_dim_* for descriptive dimension tables and warehouse_fact_* for event/measurement tables. These are the actual object names visible in the implemented Power BI model and should not be renamed in the source-to-target mapping appendix unless the physical database

objects are renamed as well. The overall model is a multi-fact dimensional model, not a single fact-table star. Verify relationship direction and cardinality in Power BI before final submission.

## 10.3 Candidate logical entities

| Logical object (Power BI model name) | Visible grain / key fields | Purpose based on visible columns |
|---|---|---|
| warehouse_dim_employee | employee_key; employee_id | Employee reference attributes; visible fields include employee_name, department and job_role |
| warehouse_dim_date | date_key | Calendar attributes including full_date, day_number, month_name, month_number, quarter_number and year_number |
| warehouse_dim_department | department_key | Department lookup using department_name |
| warehouse_dim_job | job_key | Job lookup using job_title |
| warehouse_fact_employee | employee_id; department_key; job_key (visible fields) | Employee/workforce attributes and measures including age, education_level, gender, hire_date and employee_satisfaction_score; exact grain needs confirmation |
| warehouse_fact_attendance | attendance_fact_key; employee_key; attendance_date_key | Attendance status and hours_worked |
| warehouse_fact_leave | leave_fact_key; employee_key; start_date_key; end_date_key | Leave status and leave_days |
| warehouse_fact_performance_rating | rating_fact_key; performance_id; employee_key | Performance review ratings and satisfaction measures by review_date |
| warehouse_fact_recruitment | candidate_id (visible) | Recruitment candidate attributes and hiring_decision; exact grain needs confirmation |
| warehouse_fact_payroll | employee_id; department_key (visible) | Payroll salary, allowance and related compensation fields |
| warehouse_fact_identity_events | identity_event_fact_key; employee_key; event_key | Identity event type, timestamp, changed title and group attributes |
| warehouse_fact_access_control | access_fact_key; department_key; job_key (visible) | Resource, role, role family/code and manager attributes |

## 10.4 Keys and relationships

- Preserve each source natural key (for example source_employee_id) for reconciliation and traceability.
- Use surrogate keys in the warehouse where dimensional history or conformance requires them.
- Enforce uniqueness on defined natural keys at the appropriate grain.
- Use foreign-key constraints or equivalent pre-load checks to prevent orphan fact rows.

- Define how multiple assignments, rehires, concurrent jobs and transfers affect employee grain before calculating headcount.

## 10.5 Historical data

Historical handling depends on the business question. A snapshot fact preserves point-in-time measures; a slowly changing dimension preserves selected descriptive changes. Type 1 overwrite is suitable only where prior values need not be retained. Type 2 history requires effective_start_date, effective_end_date and current_record_flag (or an equivalent pattern), with overlap and gap checks. The project should select a policy per entity rather than applying one history rule to all HR data.

## 10.6 PostgreSQL implementation standards

- Use schemas to separate staging, curated, governance and audit objects where appropriate.
- Choose date, timestamp, numeric, boolean and text types based on semantics rather than storing everything as text.
- Use NUMERIC with an explicit precision/scale for financial values; never use floating-point types for exact currency calculations.
- Index join keys, frequently filtered dates and business keys after reviewing query patterns.
- Document constraints, grants, ownership, backup/recovery and migration scripts.
- Use parameterized SQL and controlled credentials; do not embed secrets in jobs or source code.

# 11. Data Lineage and Metadata Governance

Data lineage records where a dataset originated, how it was transformed, where it was stored and which reports depend on it. EHCMDP uses four complementary lineage levels: system-level, dataset/table-level, field/column-level and run-level lineage.

## 11.1 Lineage levels

| Level | Question answered | Evidence |
|---|---|---|
| System-level | Which platforms move data between source and consumption? | Architecture diagram, system inventory and integration flow |
| Dataset/table-level | Which source datasets feed each target table or report? | Source-to-target matrix, target object register and dependency map |
| Field/column-level | How is each target column derived? | Source column, target column, transformation, datatype, null/default and key rule |
| Run-level | Which execution produced this specific load? | Run ID, batch ID, job version, timestamps, counts, status and exception log |

## 11.2 Lineage metadata register

| Metadata category | Minimum attributes |
|---|---|
| Asset identity | Asset ID, asset name, asset type, source system, owner and classification |
| Technical mapping | Source object/field, target object/field, transformation rule, datatype and key role |
| Execution | Job name/version, run ID, batch ID, start/end timestamps and status |
| Quality | Rule IDs, passed/failed counts, severity, exception reference and reconciliation result |
| Dependencies | Upstream assets, downstream tables, semantic model, report/dashboard and owner |
| Change control | Mapping version, change ticket/reference, approval, effective date and rollback note |

## 11.3 Field-level mapping standard

Every mapped target field should identify its source expression, target type, transformation, null/default behavior, validation rule and sensitivity. Derived fields must include a reproducible expression or business rule. If a field has no direct source because it is a generated key or audit field, its generation rule must be documented.

| Mapping example | Source / expression | Target rule | Quality control |
|---|---|---|---|
| employee_key | Warehouse key lookup based on approved employee identity resolution | Surrogate key in dimension/fact relationship | No duplicate dimension key; no unresolved foreign key |
| source_employee_id | Employee master source ID | Preserve as source natural key | Required, non-empty and unique within source scope |
| hire_date | Source hire-date field | Convert to DATE after source-format validation | Reject invalid date; apply approved timezone/date interpretation |
| employment_status | Source status code | Map to standardized controlled vocabulary | Unmapped code raises an exception |
| snapshot_date_key | Approved snapshot date | Join to dim_date date_key | Every published fact row resolves to a date key |
| ingestion_timestamp | Pipeline-generated UTC timestamp | Store with timezone-aware semantics where configured | Required for every ingested record/run |

## 11.4 End-to-end lineage rules

- Every target dataset must have a documented upstream source and transformation owner.
- Every critical KPI must map to a target measure and a business definition.
- Every scheduled run must be identifiable through a stable run or batch ID.
- Changes to source columns must trigger impact assessment for mappings, SQL, tests and reports.
- Lineage diagrams must be versioned with the implementation and clearly marked logical or physical.

## 11.5 Diagram interpretation

The diagrams included in this report are logical design views aligned with the stated toolchain. They are not screenshots of a live deployed environment. Physical job IDs, schema names, target object names and dashboard names should match the actual project artifacts when the implementation is reviewed.

# 12. Data Quality and Reconciliation

Data quality is implemented as explicit gates rather than a one-time profiling activity. Rules should be assigned an identifier, severity, target, execution stage, threshold, owner and disposition. Critical failures block publication; warning-level exceptions may proceed only under an approved tolerance.

## 12.1 Core quality dimensions

| Dimension | Example rule | Typical disposition |
|---|---|---|
| Completeness | Required employee ID and snapshot date are populated | Reject/quarantine missing required key |
| Uniqueness | Natural key is unique at the documented table grain | Deduplicate only with deterministic, documented precedence; otherwise quarantine |
| Validity | Hire date is valid; status belongs to allowed code set | Reject invalid values or map through approved reference data |
| Consistency | Employee foreign key resolves; end date is not before start date | Block affected records and report relationship failure |
| Timeliness | Extract and load timestamps meet agreed freshness window | Warn or fail according to business criticality |
| Accuracy / reconciliation | Source and target counts or control totals reconcile | Investigate differences; do not silently suppress them |
| Integrity | Fact rows reference valid dimensions | Block orphan rows from curated publication |

## 12.2 Data quality exception flow

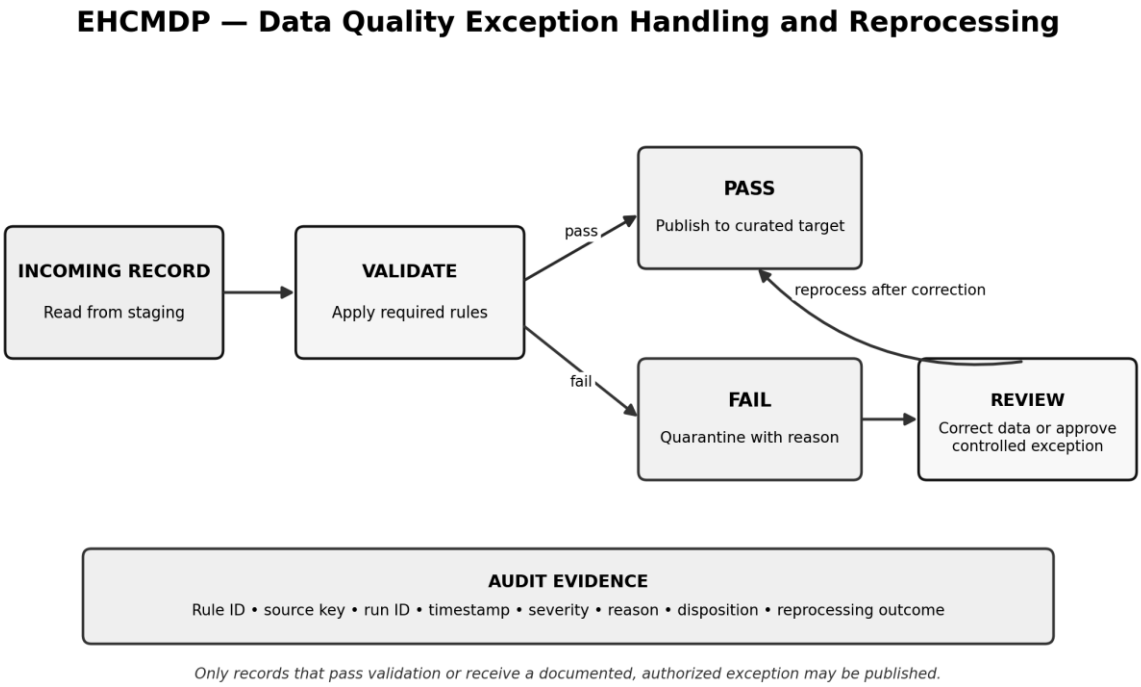



*Figure 4. Quality exception lifecycle with controlled review and reprocessing.*

## 12.3 Reconciliation controls

- Source count = accepted count + rejected count, adjusted only for documented filtering or aggregation rules.
- Target count agrees with accepted input at the target grain after documented deduplication and merge behavior.
- Distinct natural-key counts are compared before and after transformation.
- Control totals such as payroll amounts are reconciled only where source totals, currency and aggregation grain are compatible.
- Date-range coverage and missing-period checks are run for periodic datasets.
- Power BI totals are compared with curated PostgreSQL queries using the same filters, date context and metric definition.

## 12.4 Quarantine and reprocessing

Quarantined records should retain source key, batch/run ID, rule ID, failure reason, detected timestamp and disposition. Corrected records must be reprocessed through the controlled pipeline and linked to the original

exception. Manual overrides require an approver, rationale and audit entry. Exceptions should not be silently deleted or manually edited in the curated target.

## 12.5 Quality rule catalogue

| Rule ID | Rule | Severity | Action |
|---|---|---|---|
| DQ-001 | Required source employee identifier is present | Critical | Quarantine record |
| DQ-002 | Source natural key is unique at documented grain | Critical | Quarantine duplicates or apply approved deterministic resolution |
| DQ-003 | Dates parse correctly and satisfy start/end ordering | High | Reject invalid dates; retain failure reason |
| DQ-004 | Controlled codes map to an approved reference value | High | Quarantine unmapped code |
| DQ-005 | Fact foreign keys resolve to required dimensions | Critical | Block publication of orphan facts |
| DQ-006 | Source/accepted/rejected/target counts reconcile | Critical | Fail or hold run pending investigation |
| DQ-007 | Load freshness is within approved operating window | Medium/High | Warn or hold publication by dataset criticality |
| DQ-008 | Sensitive fields are not exposed to unauthorized roles | Critical | Block release and investigate access configuration |

## 12.6 Validation results available from PostgreSQL

The supplied Data Validation Report records the latest available validation outputs for EHCMDP_PostgreSQL. These are development-time results rather than a live database-generated report; the SQL validation script should be rerun before final submission. The figures below are reported as supplied and should not be interpreted as proof of a fresh run.

| Warehouse table | Reported rows | Validation interpretation |
|---|---|---|
| dim_date | 3,486 | Calendar dimension populated |
| dim_department | 13 | Department dimension |

| | | |
|---|---|---|
| | | populated |
| dim_employee | 793 | Employee dimension populated |
| dim_job | 44 | Job dimension populated |
| fact_access_control | 98,307 | Access-control records loaded |
| fact_attendance | 100 | Attendance fact populated; compare with source grain |
| fact_employee_performance | 100,000 | Performance fact populated |
| fact_identity_events | 612 | Identity event records loaded |
| fact_leave | 902 | Leave fact populated |
| fact_payroll | 3,214 | Payroll fact populated |
| fact_performance_rating | 20,127 | Performance-rating fact populated |
| fact_recruitment | 120,001 | Recruitment fact populated |

The validation report records zero missing and zero unmatched keys for the performance-rating date-key check (20,127 rows) and the recruitment job-key check (120,001 rows). The performance-rating date-key check is marked PASS. The foreign-key orphan check reports zero orphans for the listed warehouse relationships, including employee, date, department and job references.

The staging row-count snapshot also identifies objects with zero rows, including stg_hr_analytics, stg_hr_saas_snapshots_cle... and stg_performance_eval. These require investigation before claiming complete ingestion. Other reported staging counts differ from the source inventory in places; confirm whether this reflects repeated loads, filtered records, different extracts or target-table grain, and document the explanation.

**Table 12.1. Warehouse row counts reported by the supplied validation report.**

# 13. Security, Privacy and Access Control

Human capital data may include direct identifiers, compensation, performance, attendance and access-control information. The platform should apply data minimization, purpose limitation, least privilege and auditable access. Legal and privacy stakeholders must determine applicable jurisdictional obligations and retention periods.

## 13.1 Security controls

| Control area | Required design practice |
|---|---|
| Authentication | Use approved account management; separate individual and service identities |
| Authorization | Role-based permissions for ingestion, staging, curated data, governance metadata and BI |
| Least privilege | Grant only the tables, columns and actions needed for each role |
| Sensitive columns | Restrict or mask names, contact details, payroll, performance and leave-related attributes as appropriate |
| Credential handling | Use environment configuration or a secret manager; never commit passwords or tokens |
| Transport and storage | Use approved encrypted connections and protected storage according to organizational policy |
| Audit logging | Log access and changes to sensitive data, roles, mappings and production configurations |
| Data export | Restrict exports and sharing; review Power BI workspace and row-level security configuration |
| Retention and deletion | Apply approved retention schedules, archival, legal hold and deletion processes |
| Backup and recovery | Protect backups with access control and test restoration according to recovery requirements |

## 13.2 Privacy by design

- Use only fields required for the stated analytical purpose.
- Prefer aggregated reporting where employee-level detail is not required.
- Separate identity-resolution attributes from broad analytical datasets where feasible.
- Use synthetic or masked data for demonstrations and screenshots.

- Review whether a KPI could expose sensitive individual information in small groups.
- Document the purpose, owner, classification and approved audience for each dataset.

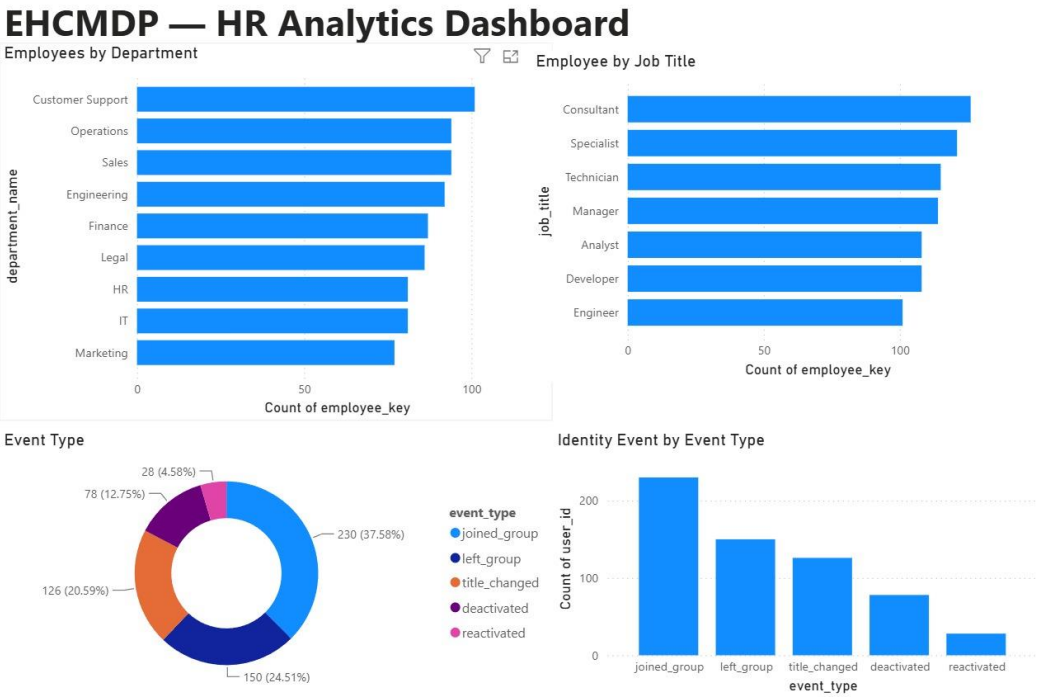
## 13.3 Access roles

| Role | Typical permissions |
|---|---|
| Pipeline service account | Read approved source extracts; write landing/staging and curated targets required by jobs |
| Data engineer | Develop jobs and schemas; restricted access to sensitive production data; controlled deployment permissions |
| Data steward | Read quality results and exceptions; approve defined dispositions; no broad administrative access by default |
| BI analyst | Read approved curated views and semantic model; no direct access to unnecessary sensitive columns |
| HR authorized consumer | Access dashboards and detail consistent with approved HR role and reporting purpose |
| Database administrator | Administer platform under privileged access logging and change control |

# 14. Analytics and Power BI Consumption

## 14.1 Power BI Dashboard Evidence

Dashboard evidence: insert the final Power BI screenshot after confirming that the visuals, filters, KPI values and refresh status match the validated model.




## 14.1 Reporting design

Power BI should consume curated PostgreSQL datasets or approved views rather than independently reimplementing source cleanup logic in multiple reports. Semantic-model measures should align with the business glossary and should be validated against the same source grain and filters used in database checks.

## 14.2 Candidate workforce metrics

| Metric | Definition / formula | Important interpretation |
|---|---|---|
| Headcount | Count distinct employees meeting the approved employment-status and snapshot-date rules | Define whether contractors, contingent workers and concurrent assignments are included |
| FTE | Sum of approved full-time-equivalent values at the defined snapshot grain | Confirm whether values are capped, prorated or assignment-based |

| | | |
|---|---|---|
| Hires | Count of qualifying hire events in the reporting period | Use an approved hire event/date and treatment for rehires |
| Attrition rate | Qualifying separations during period ÷ approved average headcount for period × 100 | Define separation population, period boundaries and average-headcount method |
| Absence / leave rate | Approved absence or leave measure ÷ approved scheduled-work denominator | Use only a defined, privacy-reviewed denominator and leave classification |
| Learning completion rate | Employees or enrollments completed ÷ eligible enrolled population × 100 | State whether the denominator is employees, courses or enrollments |
| Recruitment conversion | Count reaching a defined funnel stage ÷ count entering the relevant prior stage × 100 | Specify funnel stage, cohort and date attribution |
| Payroll total | Sum of approved payroll amount for a defined period and currency | Do not aggregate currencies without an approved conversion rule |

## 14.3 Dashboard quality checks

- Every page identifies reporting period, refresh timestamp and relevant filters.
- Totals reconcile to curated PostgreSQL queries for the same date context and population.
- Metric definitions are documented and consistent across pages.
- Sensitive detail is restricted by workspace permissions and row/column-level controls where required.
- Missing data, stale refreshes and suppressed small-group results are communicated clearly.

## 14.4 Consumption lineage

For each dashboard measure, the lineage register should identify the Power BI semantic-model table/measure, its PostgreSQL source view/table, the source domain and the transformation or business rule that produced it.

Dashboard refresh status should be linked to the underlying data load status so that a successful visual refresh is not mistaken for a successful upstream data load.

# 15. Testing and Validation Strategy

Testing should prove that the pipeline preserves data meaning and makes failures visible. Tests should be repeatable and retained with run metadata, expected outcomes and actual results. This report defines the test plan; actual pass/fail outcomes should be reported only after the tests are executed.

| Test ID | Test scenario | Expected result / evidence |
|---|---|---|
| T-01 | Valid source extract is ingested | Run is logged; row counts and batch identity are captured |
| T-02 | Required column is missing | Structural validation fails before promotion; error identifies missing column |
| T-03 | Invalid date or unmapped code is present | Record is rejected/quarantined with rule ID and reason |
| T-04 | Duplicate natural key occurs | Duplicate is detected; deterministic approved handling or quarantine is applied |
| T-05 | Foreign key does not resolve | Orphan record is blocked from curated publication |
| T-06 | Job is rerun for same batch | Idempotency controls prevent unintended duplicate target rows |
| T-07 | Source and target counts differ | Reconciliation raises an exception and run is held or failed per policy |
| T-08 | Source schema changes | Change is detected and downstream impact is reviewed before release |
| T-09 | Unauthorized role requests sensitive column | Access is denied and logged where audit logging is configured |
| T-10 | Power BI measure is compared with SQL result | Values match under identical metric definition, filters and date range |
| T-11 | A failed load is reprocessed after correction | New run links to original exception; result and disposition are auditable |
| T-12 | Backup/restore procedure is exercised | Restoration is verified against approved recovery criteria |

## 15.1 Test evidence package

- Versioned test case and expected outcome.
- Source batch/sample identifier and masking status.
- Job version, run ID, start/end timestamps and final status.

- Source, accepted, rejected and target counts.
- SQL validation output, quality results and exception references.
- Power BI reconciliation evidence where applicable.
- Defect record, remediation and retest result for failed cases.

Validation evidence to retain: the SQL script version and execution timestamp, staging and warehouse row-count outputs, foreign-key/orphan-check output, key-mapping checks, and a rerun result showing whether previously empty staging objects have been resolved. The supplied validation report is a snapshot; refresh these outputs before final acceptance.

# 16. Operations and Change Management

## 16.1 Operational monitoring

| Monitor | What to inspect | Response |
|---|---|---|
| Job health | Run status, failure code, duration and last successful run | Alert owner; inspect logs; restart only after failure cause is understood |
| Freshness | Source extract time and target load time | Hold stale reporting or label it clearly; investigate source or pipeline delay |
| Volume drift | Record count and key distribution versus expected range | Investigate source change, partial extract or legitimate workforce movement |
| Quality drift | Nulls, duplicates, invalid codes and rejected percentage | Assess severity; quarantine affected records and engage data owner |
| Reconciliation | Source/accepted/rejected/target counts and control totals | Hold publication for unexplained differences |
| Access/security | Role changes, failed access and sensitive data exports | Review authorization and audit trail; remove excess access |

## 16.2 Change management

1. Register the proposed change, business reason, affected source/target assets and owner.
2. Perform impact analysis across source schemas, Pentaho jobs, PostgreSQL tables, mappings, tests, glossary definitions and Power BI measures.
3. Update mapping and metadata versions; retain a reviewable change record.
4. Run regression, quality, reconciliation and security tests in a controlled environment.
5. Deploy through the approved release process with a rollback or recovery plan.
6. Confirm post-deployment lineage, refresh status and downstream report results.

## 16.3 Backup and recovery

Backup frequency, recovery point objective (RPO), recovery time objective (RTO), retention and restore testing must be agreed with the platform owner. The project should document database backup scope, configuration backup,

job/script versioning and a tested restoration procedure rather than assuming that a successful backup automatically proves recoverability.

# 17. Risks, Assumptions and Limitations

| Risk / assumption | Potential impact | Mitigation / treatment |
|---|---|---|
| Source schemas differ or change | Failed jobs, missing fields or incorrect mappings | Maintain schema contracts, detect drift and require change-impact review |
| Inconsistent employee identity | Duplicate or misjoined employee records | Define cross-system identity resolution and preserve source keys |
| Incomplete or late extracts | Stale metrics and count mismatches | Record extract timestamps, freshness checks and explicit late-arrival handling |
| Sensitive HR information exposed | Privacy and trust impact | Apply least privilege, masking, export restrictions and audit logging |
| Ambiguous KPI definitions | Conflicting reports and stakeholder disagreement | Approve glossary definitions and use shared semantic measures |
| High volume or long history | Slow loads and dashboard queries | Profile volume, index measured workloads and consider incremental loads/partitioning |
| Unapproved manual corrections | Loss of reproducibility and auditability | Use quarantine/disposition workflow and log approvals |
| Physical implementation evidence not supplied in the brief | Cannot assert actual run performance or deployment status | Report logical design and acceptance criteria; use real job/schema/run evidence during review |
| Legal retention requirements vary by jurisdiction | Over-retention or premature deletion | Obtain approved retention schedule from legal/privacy stakeholders |

## 17.1 Assumptions

- Relevant source data can be accessed through approved extracts or connections.
- HR domain owners can clarify definitions, identifiers, code sets and allowed uses.
- PostgreSQL is the designated relational platform for curated analytical storage.
- Pentaho PDI, Python/Pandas, Git/GitHub and Power BI are the stated project toolchain.
- Dashboard access and employee-level detail are subject to organizational security and privacy approval.

## 17.2 Limitations

This report is a consolidated project design and delivery specification based on the available UC17 context. It does not invent physical source application names, deployed schema names, actual job execution counts, measured performance, production acceptance results or live dashboard screenshots. The logical mappings and model are intended to guide and evaluate implementation; final evidence should reflect the artifacts and test results actually produced by the team.

# 18. Expected Outcomes and Success Criteria

The expected outcome is a governed and traceable workforce analytics foundation that reduces manual reconciliation and makes the path from source records to business measures understandable. Success should be evaluated against evidence, not assumed from the presence of tools or diagrams.

| Success area | Acceptance criterion |
|---|---|
| Ingestion | Each in-scope source has a documented extraction/load path and run identity |
| Traceability | Critical target fields and KPIs have source-to-target mapping and transformation rules |
| Data quality | Critical rules execute and produce retained pass/fail outcomes |
| Reconciliation | Source, accepted, rejected and target differences are explained and dispositioned |
| Storage | Curated structures have documented grain, keys, relationships and history policy |
| Security | Roles, sensitive columns, credential handling and audit controls are reviewed |
| Analytics | KPI definitions are approved and Power BI values reconcile to curated queries |
| Operations | Runbook, failure response, change process and recovery approach are documented |
| Maintainability | Code, jobs, mappings and tests are versioned and change impact is traceable |

## 18.1 Project completion evidence

A complete review package should include the project repository, Pentaho job exports or screenshots, PostgreSQL DDL and representative query results, profiling/quality outputs, run logs, lineage mapping register, Power BI report or screenshots, test evidence and this report. Evidence must be genuine and consistent with the implementation state.

# 19. Conclusion

The EHCMDP project defines a structured approach for consolidating HR data domains into a governed workforce analytics platform. Its main contribution is the integration of data engineering with explicit lineage, quality, security and operational controls. Pentaho PDI provides the orchestration pattern, PostgreSQL provides the relational and analytical storage foundation, Python/Pandas supports profiling and validation, Git/GitHub supports versioned change management, and Power BI provides the consumption layer.

The report establishes a logical architecture, source inventory, ingestion lifecycle, warehouse design pattern, source-to-target mapping standard, lineage metadata requirements, quality rule catalogue, privacy controls, KPI definitions, testing strategy and operational acceptance criteria. The resulting framework supports reproducibility and auditability while making clear which elements are design specifications and which require implementation evidence from actual project artifacts.

# Appendix A. Source-to-Target Mapping Register

Field-level alignment note: targets are listed using the actual table/column labels visible in the supplied screenshot. The screenshot shows only a subset of columns in each table, so the register documents visible fields and flags uncertain object names/grain for verification; it does not claim to be a complete database catalog.

Mapping convention: Appendix A uses the table names visible in the supplied Power BI Model view: warehouse_dim_employee, warehouse_dim_date, warehouse_dim_department, warehouse_dim_job, warehouse_fact_employee, warehouse_fact_attendance, warehouse_fact_leave, warehouse_fact_performance_rating, warehouse_fact_recruitment, warehouse_fact_payroll, warehouse_fact_identity_events and warehouse_fact_access_control. Field names are transcribed from the visible columns in the screenshot; fields not visible in the screenshot are not assumed. The employee fact table name is partially truncated in the image, so verify its exact physical name. The listed fact grain and key constraints must be confirmed against PostgreSQL and Power BI before release.

| Source domain | Source field / entity | Target object / field | Transformation / rule | Validation |
|---|---|---|---|---|
| HRMS / Employee Master | Employee ID | warehouse_dim_employee.employee_id; warehouse_dim_employee.employee_key | Preserve source employee ID; resolve employee_key using approved identity lookup | employee_id required; employee_key unique and non-null |
| HRMS / Employee Master | Employee name | warehouse_dim_employee.employee_name | Trim whitespace; standardize casing only where approved | Name handling follows approved privacy/access policy |
| HRMS / Employee Master | Department | warehouse_dim_employee.department; warehouse_dim_department.department_name | Standardize source department and map to department lookup | Department lookup match; no orphan department keys |
| HRMS / Employee Master | Job role / title | warehouse_dim_employee.job_role; warehouse_dim_job.job_title | Normalize title and resolve job_key | No unresolved job lookup for published rows |
| HRMS / Employee Master | Age, gender, education level, hire date, satisfaction | warehouse_fact_employee.age; gender; education_level; hire_date; employee_satisfaction_score | Cast numeric/date fields; standardize controlled categories; preserve permitted source values | Valid age/date ranges; satisfaction within defined scale |
| Calendar source / derived | Full date, day, month, quarter, year | warehouse_dim_date.full_date; day_number; month_name; month_number; quarter_number; year_number | Generate calendar attributes from full_date; use one | date_key unique; calendar |

| | | | | |
|---|---|---|---|---|
| calendar | | | row per date | attributes consistent with full_date |
| Attendance source | Employee ID, attendance date, status, hours | warehouse_fact_attendance.employee_key; attendance_date_key; attendance_status; hours_worked | Resolve employee/date keys; standardize status; convert hours to agreed unit | Keys resolve; hours non-negative; valid status |
| Leave source | Employee ID, start/end dates, days, status | warehouse_fact_leave.employee_key; start_date_key; end_date_key; leave_days; leave_status | Resolve employee/date keys; normalize duration and status | End date not before start date; leave_days non-negative |
| Performance rating source | Employee ID, performance ID, review date, ratings | warehouse_fact_performance_rating.employee_id; employee_key; performance_id; review_date; environment_satisfaction; job_satisfaction; relationship_satisfaction; manager_rating; rating_fact_key | Resolve employee key; parse review date; cast rating values to numeric; preserve performance_id | Required identifiers; valid review date and approved rating scales |
| Recruitment source | Candidate ID, aptitude/communication scores, education, experience, expected salary, decision | warehouse_fact_recruitment.candidate_id; aptitude_test_score; communication_score; education_level; experience_years; expected_salary; hiring_decision | Cast score/experience/salary fields; normalize hiring-decision vocabulary | Candidate key present; valid score ranges; decision maps to approved values |
| Payroll source | Employee ID, salary, allowance, country, business unit, department, designation | warehouse_fact_payroll.employee_id; basic_salary; allowance; country; business_unit; department; designation; department_key | Use decimal types for money; retain currency/period fields if present; map department_key | Amounts valid; period/currency requirements checked; reconcile approved totals |
| Identity / directory events | Event ID, employee ID, event timestamp/type, title, group | warehouse_fact_identity_events.identity_event_fact_key; event_key; employee_key; event_timestamp; event_type; changed_title; group_id; group_name; ingested_at | Parse timestamp; resolve employee key where applicable; record ingestion timestamp | Event key and timestamp valid; event type controlled |
| Access control source | Resource, manager, role, department, job | warehouse_fact_access_control.access_fact_key; resource; manager_id; role_code; role_family; department_key; department_name; job_key; job_title | Normalize role fields; resolve department/job keys where applicable | Access key unique; role/resource fields meet requiredness rules |
| Pipeline runtime | Run ID / batch ID / timestamps | Not visible in supplied model — confirm audit schema/table | Generate stable execution identifiers and persist run metrics in a separately confirmed audit target | Run record exists for every attempted load |

# Appendix B. Data Quality Rule Catalogue

| Rule ID | Field / asset | Condition | Severity | Failure evidence |
|---|---|---|---|---|
| DQ-001 | Employee identifier | Not null and not blank | Critical | Source key, run ID, rejected row and reason |
| DQ-002 | Natural key | Unique at documented entity grain | Critical | Duplicate key group and resolution disposition |
| DQ-003 | Date fields | Parseable and valid for business context | High | Original value, normalized value and parse error |
| DQ-004 | Status / code fields | Member of approved reference set | High | Unmapped code and source context |
| DQ-005 | Foreign keys | All required dimension references resolve | Critical | Missing dimension key and fact reference |
| DQ-006 | Load counts | Source = accepted + rejected under documented rules | Critical | Counts, variance and reconciliation status |
| DQ-007 | Target counts | Target changes align with load/merge policy | Critical | Before/after count and duplicate check |
| DQ-008 | Freshness | Load age within approved dataset threshold | Medium/High | Source timestamp, load timestamp and measured lag |
| DQ-009 | Sensitive access | Only approved roles can read restricted columns | Critical | Role/grant review and access audit evidence |
| DQ-010 | Power BI reconciliation | Metric equals SQL reference under same filters | High | Measure, filter context, SQL query and comparison result |

# Appendix C. Business Glossary and KPI Definitions

| Term / KPI | Definition | Calculation / interpretation note |
|---|---|---|
| Employee | A person represented in the approved employee identity model | Cross-system identity resolution rules determine whether records represent the same person |
| Headcount | Number of distinct qualifying employees at a defined point in time | Specify included worker types, employment statuses and snapshot date |
| FTE | Full-time-equivalent workforce measure | Use approved source value or documented hours-to-FTE conversion |
| Hire | Qualifying employment commencement event | Define rehire and transfer treatment |
| Separation | Qualifying employment termination event | Define eligible termination reasons and effective date |
| Attrition rate | Separations relative to the approved average workforce base | Specify period, denominator and treatment of internal transfers |
| Leave duration | Duration of an approved leave event | Define calendar days versus working days and partial-day treatment |
| Learning completion | A course enrollment meeting the approved completion rule | Define completion status and denominator |
| Recruitment conversion | Movement between defined hiring funnel stages | Define cohort date, stage transition and denominator |
| Data lineage | Traceable path from source asset through transformations to output | Includes system, dataset, field and run-level evidence |
| Quarantine | Controlled holding area for records failing defined validation rules | Requires reason, rule ID, ownership and disposition |
| Batch / run | A uniquely identified processing execution or source delivery | Use consistent identifiers in logs, target audit metadata and exception records |

# Appendix D. Lineage Evidence Checklist

- Source system, source object, owner and sensitivity classification are documented.
- Extraction method, snapshot timestamp and source schema version are recorded.
- Pentaho job name/version and run/batch identifiers are available.
- Every critical target table has an upstream dependency and mapping register.
- Every critical target field has a source expression or generation rule.
- Transformation rules, defaults, data types, keys and null handling are documented.
- Quality rule results and exception dispositions are retained.
- Source, accepted, rejected and target counts reconcile or have approved explanations.
- PostgreSQL target structures, constraints and access grants are documented.
- Power BI measures are linked to their curated source and approved KPI definitions.
- Change impact, test evidence, release approval and rollback notes are retained.
- Diagrams are versioned and clearly distinguish logical design from physical evidence.